

Pemrograman Web Lanjut

SIB245007 | D-IV Sistem Informasi Bisnis

Pertemuan 9: **Pengolahan Data: Impor, Ekspor, dan Antrean**

Laporan dengan filter tanggal, ekspor dan impor CSV, dan kapan pekerjaan berat sebaiknya dipindah ke latar belakang

Yang Akan Kamu Pelajari

1. Menyusun laporan yang mengagregasi data dalam rentang tanggal, dan menghindari jebakan umum saat memfilter tanggal
2. Mengekspor data ke CSV tanpa membebani memori server, dan mengimpor CSV dengan menangani baris yang cacat tanpa menghentikan seluruh proses
3. Mengenali kapan sebuah proses sebaiknya dipindah ke **antrean (queue)** alih-alih dijalankan langsung dalam siklus request

Slide ini membahas konsep. Menerapkan laporan, ekspor/impor, dan antrean pada Simple POS dikerjakan di jobsheet praktikum.

Bagian 1

Laporan dan Agregasi Data

Empat Istilah Dasar

ETL (Extract, Transform, Load): pola umum mengambil data mentah, mengubah bentuknya, lalu menyimpan atau menyajikannya dalam bentuk baru.

Batch Processing: memproses banyak data sekaligus dalam kelompok, bukan satu per satu.

Antrean (Queue): mekanisme menunda pekerjaan berat agar dijalankan di latar belakang oleh proses terpisah, bukan langsung selama request pengguna berlangsung.

Import/Export: membawa data masuk dari format eksternal (CSV, Excel) ke basis data, atau mengeluarkannya ke format eksternal untuk dibaca di luar aplikasi.

Laporan Menjawab Pertanyaan yang Berbeda

Halaman daftar data biasa menjawab "apa saja yang ada": tampilkan semua artikel, semua transaksi, semua pengguna. Halaman laporan menjawab pertanyaan yang lebih sempit tapi lebih berat: "berapa total X pada rentang waktu tertentu".

```
$articles = Article::whereBetween('published_at', [  
    $from, $to,  
])->get();  
  
$totalKata = $articles->sum('word_count');
```

`whereBetween` memfilter baris yang tanggalnya berada di antara dua batas, lalu `sum()` menjumlahkan satu kolom dari hasil filter itu. Pola yang sama berlaku untuk menjumlahkan apa pun yang sudah tersimpan per baris.

Jebakan Umum: Batas Hari yang Terpotong

Filter tanggal "1 Agustus sampai 31 Agustus" yang ditulis apa adanya secara harfiah berarti "sampai jam 00:00:00 tanggal 31", yang justru mengecualikan seluruh transaksi yang terjadi sepanjang tanggal 31 itu sendiri.

```
$from = $request->date('from')->startOfDay();  
$to = $request->date('to')->endOfDay();
```

`startOfDay()` mengubah tanggal awal menjadi jam 00:00:00, dan `endOfDay()` mengubah tanggal akhir menjadi jam 23:59:59, supaya seluruh hari terakhir ikut terhitung, bukan terpotong di tengah malam.

Agregat yang Sudah Tersimpan, Bukan Dihitung Ulang

Pertemuan 6 menegaskan: nilai seperti total transaksi tidak boleh dipercaya kalau datang langsung dari input klien, dan harus dihitung ulang di server saat transaksi dibuat. Begitu nilai itu tersimpan dengan benar, laporan boleh memakainya apa adanya.

```
// Laporan cukup menjumlahkan kolom yang sudah tersimpan  
$totalPenjualan = $transactions->sum('total');
```

Menghitung ulang total dari baris detail transaksi satu per satu, setiap kali laporan dibuka, membebani server tanpa alasan: angkanya sudah benar sejak transaksi itu dibuat.

Bagian 2

Ekspor dan Impor CSV

Mengekspor Tanpa Membebani Memori

Cara paling sederhana mengekspor CSV adalah mengumpulkan semua baris jadi satu string besar, lalu mengirimkannya sekaligus. Cara itu bekerja untuk puluhan baris, tapi begitu jumlah barisnya mencapai puluhan ribu, seluruh berkas harus muat dulu di memori server sebelum terkirim sama sekali.

```
return response()->streamDownload(function () {  
    $out = fopen('php://output', 'w');  
    fputcsv($out, ['Judul', 'Penulis', 'Tanggal']);  
    foreach ($articles as $a) {  
        fputcsv($out, [$a->title, $a->author->name, $a->published_at]);  
    }  
    fclose($out);  
}, 'laporan-artikel.csv');
```

`streamDownload()` menulis setiap baris langsung ke output stream sambil mengirimkannya ke browser, tanpa menahan seluruh berkas di memori server terlebih dahulu.

Mengimpor: Baris Cacat Tidak Boleh Menghentikan Semuanya

Impor berjalan ke arah sebaliknya: membaca berkas CSV yang diunggah, lalu mengubah tiap barisnya menjadi baris baru di database. Bagian yang rawan bukan baca-tulisnya, melainkan menangani baris cacat tanpa menghentikan seluruh proses gara-gara satu baris bermasalah.

```
foreach ($rows as $row) {  
    $validator = Validator::make($row, [  
        'title' => ['required', 'string'],  
        'author_id' => ['required', 'exists:authors,id'],  
    ]);  
  
    if ($validator->fails()) {  
        continue; // lewati baris ini, lanjut ke baris berikutnya  
    }  
  
    Article::create($validator->validated());  
}
```

Bagian 3

Kapan Memakai Antrean

Dua Cara Memproses Pekerjaan

Langsung (dalam request)

- Pengguna menunggu sampai selesai
- Kalau gagal, pengguna langsung tahu
- Cocok untuk pekerjaan singkat, hasilnya dibutuhkan seketika

Lewat Antrean (job)

- Respons instan, pekerjaan berjalan di latar belakang
- Pengguna tidak menunggu di depan browser
- Cocok untuk pekerjaan berat atau berdurasi tidak pasti

Impor beberapa puluh baris selesai dalam hitungan detik, cukup dijalankan langsung. Begitu volumenya naik ke ribuan baris, proses yang sama bisa memakan waktu puluhan detik, dan permintaan HTTP punya batas waktu tunggu.

Tanda-Tanda Proses Layak Dipindah ke Antrean

- Durasinya tidak bisa diprediksi di muka, tergantung ukuran data yang diproses
- Hasilnya tidak perlu dilihat pengguna seketika di halaman yang sama
- Melibatkan panggilan ke layanan eksternal yang lambat, misalnya mengirim email atau memanggil API pihak ketiga

Laporan yang dibuka dan dibaca langsung di halaman biasanya tetap lebih baik diproses langsung, karena pengguna memang menunggu hasilnya saat itu juga. Antrean bukan solusi untuk semua pekerjaan berat, hanya untuk yang tidak butuh hasil instan.

Alur Job di Laravel

Controller



dispatch() job



Tabel jobs



Worker (queue:work)

```
php artisan make:job ProcessArticleImport
```

```
ProcessArticleImport::dispatch($csvPath);  
  
return back()->with('success', 'Impor sedang diproses di latar belakang.');
```

Controller hanya menyimpan berkas dan mengirim satu job ke antrean, lalu segera merespons pengguna, tanpa menunggu prosesnya selesai sama sekali. Pekerjaan sesungguhnya (loop validasi dan penyimpanan) dipindah ke metode `handle()` pada class job itu.

Worker: Proses Terpisah yang Menjalankan Job

Worker: proses PHP terpisah yang terus berjalan, mengambil job dari tabel `jobs` satu per satu, lalu menjalankan metode `handle()` -nya.

```
php artisan queue:work
```

Tanpa worker yang berjalan, job hanya menumpuk di tabel `jobs` dan tidak pernah diproses, karena dispatch hanya menyimpan job ke antrean, tidak menjalankannya secara langsung.

Kalau `QUEUE_CONNECTION` di `.env` bertuliskan `sync`, job dijalankan langsung di tempat tanpa pernah masuk antrean sungguhan, menghilangkan seluruh manfaat memindahkannya ke latar belakang.

Menerapkan pada Simple POS

Konsep laporan, ekspor/impor CSV, dan antrean ini akan kamu terapkan langsung pada Simple POS di jobsheet praktikum: menyusun laporan penjualan dengan filter tanggal, mengekspor laporan itu ke CSV, membangun impor produk dari CSV, lalu memindahkan impor itu ke job antrean supaya pengguna tidak menunggu di depan browser.

Kode lengkap: github.com/se-polinema/simple-pos-ch08

Rangkuman (1/2)

- Laporan mengagregasi data dalam rentang tanggal lewat `whereBetween` , dengan `startOfDay()` / `endOfDay()` supaya hari terakhir tidak terpotong di tengah malam
- Nilai yang sudah tersimpan dengan benar (seperti total transaksi) boleh dipakai apa adanya di laporan, tidak perlu dihitung ulang setiap kali laporan dibuka
- Ekspor CSV memakai `streamDownload()` untuk menulis baris langsung ke output tanpa menahan seluruh berkas di memori

Rangkuman (2/2)

- Impor CSV memvalidasi tiap baris satu per satu dan melewati baris yang gagal lewat `continue` , bukan menghentikan seluruh proses
- Proses berdurasi tidak terprediksi atau bervolume besar sebaiknya dipindah ke job antrean, supaya pengguna mendapat respons instan sementara prosesnya berjalan di latar belakang
- Job yang di-dispatch hanya masuk tabel `jobs` , dan baru benar-benar berjalan setelah worker (`queue:work`) memprosesnya

Referensi & Diskusi

Dokumentasi resmi Laravel (Queues, File Storage, HTTP Responses)

Kode lengkap: `github.com/se-polinema/simple-pos-ch08`

Pertemuan berikutnya: Merancang dan Membangun REST API